Makeen at AISA-ArabicFC: Grounding LLM via Extracted Spans and Generalization Trade-off in Arabic Function Calling

Nouf Alzahrany*
King Saud University
Saudi Arabia
Nalzahrany0016@gmail.com

Abir Korched*
Independent Researcher
Tunisia
korched.abir@gmail.com

Bassam Mattar*
Alexandria University
Egypt
b.mattar@alexu.edu.eg

Abstract

Reliable function calling is challenging for Arabic due to dialectal variation and limited task-specific resources. We present **Makeen**, a data-centric approach for the AISA-ArabicFC shared-task, and extend it with span extraction and confidence-gated routing. The best post-submission variant achieves 83.20% argument exact match and 91.11% Track B on development data under held-out (cross-validated) routing, while the submitted model achieves 82.65% argument exact match and 91.06% Track B on the blind test set, ranking 9th in Task B. We also release two Arabic tool-use datasets and show that external-tool augmentation improves unseen-tool argument exact match from 44.49% to 66.53%. These results highlight the importance of data quality, argument grounding, and tool-level generalization for Arabic function calling.

1 Introduction

Function calling refers to the capability of a large language model (LLM) to access predefined external functions and transform natural-language instructions into structured requests (Wang et al., 2026). Reliable function calling requires a precise understanding of user intent and accurate extraction of function arguments. For Arabic-language systems, this process is further complicated by orthographic variation (Darwish et al., 2021), limited Arabic function-calling resources (Ersoy et al., 2025), and the diversity of Arabic dialects (Nacar et al., 2026b).

We present **Makeen**, our system for Arabic function calling. The submitted system is a generator fine-tuned on filtered data and enhanced tool declarations (§4). We extend it with extractive hints and confidence-gated routing (§3), and report both stages. Our main contributions are:

- A hybrid system pairing a label-semantic span extractor with a hint-grounded generator.
- Confidence-gated per-argument routing between the two components.
- **Public release of Arabic function-calling resources** addressing the scarcity of publicly available Arabic data: translations of two English datasets with tool declarations and reasoning traces, providing 5,758 examples across 1,442 distinct tools. [1] [2]
- A study of generalization to unseen tools (§4.5), evaluating systems both on the shared-task dev split of 27 tools and on an unseen-tool split held out from the translated data.

2 Background

2.1 Task And Data

The AISA-ArabicFC shared-task (Nacar et al., 2026a) requires mapping an Arabic utterance onto a single tool call. Each instance presents four candidate declarations, sampled from an inventory of 27 tools across eight real-world domains: at most one is correct, the remainder being distractors, and a system either selects that tool and predicts its arguments or abstains with `none` (Figure 1, Appendix A). This work addresses Track B, which additionally requires an Arabic `<think>` reasoning trace and is scored

$$0.3\,\textsc{FnAcc} + 0.5\,\textsc{ArgEM} + 0.2\,\textsc{ThinkRate} \quad (1)$$

with ArgEM satisfied only by an exact match over all arguments. The released AISA-ArabicFC data (Nacar et al., 2026b,c,a) cover five Arabic dialects, comprising 10,550 training, 545 development and 1,125 blind test instances.

* These authors contributed equally to this work.

[1] https://huggingface.co/datasets/Makeen-AraFC/hermes-tool-use-reasoning-ar

[2] https://huggingface.co/datasets/Makeen-AraFC/glaive-tool-use-reasoning-ar

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1635–1653
October 28-29, 2026 ©2026 Association for Computational Linguistics

2.2 Related Work

Arabic function-calling resources remain comparatively limited, particularly for dialectal and multilingual settings. Ersoy et al. (Ersoy et al., 2025) studied data strategies and instruction tuning for Arabic tool calling and showed the value of translating and adapting English tool-use datasets. More recently, AISA-ArabicFc introduced a larger Arabic function-calling resource with dataset auditing, schema normalization, and tool-aware fine-tuning (Nacar et al., 2026c). We build on this line of work in a shared-task setting with noisy supervision, refined tool declarations, and explicit evaluation of generalization to held-out tools. Our approach further uses a hybrid generative-extractive architecture to improve exact argument matching.

3 System Overview

Given an Arabic utterance q and candidate tool declarations $\mathcal{T}$, Makeen predicts a tool $\hat{y} \in \mathcal{T} \cup \{\texttt{none}\}$ and an argument map $\hat{a}$. The submitted system, $\text{Makeen}_{\text{Base}}$, is a generator fine-tuned on the filtered and enhanced shared-task data. We then extend it in two stages: first, $\text{Makeen}_{\text{+Hint}}$ augments the generator with a label-semantic extractor E that supplies structured hints; second, confidence-gated routing combines the generator and extractor outputs at inference time. The complete Makeen pipeline is shown in Figure 7(Appendix J).

The two components are complementary: the extractor preserves surface-grounded values exactly, while the generator can normalize values or infer arguments that are not explicitly expressed in the utterance. Routing exploits this complementarity by selectively replacing generated arguments with confident extracted values.

3.1 Label-Semantic Extractor

E is a bi-encoder initialised from `multilingual-e5-base` (Wang et al., 2024; Conneau et al., 2020); bi-encoders have been applied to Arabic span extraction before, notably for named-entity recognition (Hussein et al., 2023). Following label-semantic tagging (Ma et al., 2022), tools and arguments are represented by their natural-language descriptions rather than class indices, so declarations unseen in training remain scorable; candidates are re-sampled per example. The utterance is encoded *once* with offset mapping retained: mean-pooling all token states gives e_q, and mean-pooling those overlapping word w_i gives a contextual e_{w_i}, so one pass serves both objectives.

Tool scores $s(t) = e_q^\top e_t/\tau$ are read against a no-call score $s_\varnothing = e_q^\top e_\varnothing/\tau$, at the training temperature $\tau = 0.05$: E calls no tool when $\max_t s(t) < \theta$ or $s_\varnothing > \max_t s(t)$, with $\theta = 3.0$ throughout. Words are scored against the selected tool's arguments, $S_{i,p} = e_{w_i}^\top e_p/\tau$, and decoded by *exclusive assignment*

$$\pi(i) = \arg\max_p S_{i,p}, \quad \text{kept iff } S_{i,\pi(i)} > \theta, \quad (2)$$

so each word is claimed by at most one argument. A value is the concatenation of its words, and $c_p = \max_{i:\pi(i)=p} S_{i,p}$ is the confidence the router consumes. Training sums InfoNCE over the in-batch candidates including NONE (van den Oord et al., 2018) with a per-word binary cross-entropy against each argument's gold span, positives up-weighted by $n_{\text{neg}}/n_{\text{pos}}$ to offset the outside-label imbalance.

3.2 Hint-Grounded Generation

The generator is Qwen3.5-4B (Yang et al., 2025) adapted with LoRA (Hu et al., 2021). For $\text{Makeen}_{\text{+Hint}}$, the extractor output is serialized into a three-line Arabic hint block appended to the user turn:

```
الأداة المتوقعة: check_traffic_violations
المُستخرج: id_number=1122334455
يحتاج استنتاج: plate_number
```

giving the predicted tool, the arguments already grounded in surface text, and those still to be inferred; negatives carry الأداة المتوقعة: `none`. The third line makes the block a *plan* rather than an answer. Hints are injected identically at training and test time. Training hints are perturbed (§4) so that the generator learns to tolerate and repair extractor errors. The block is therefore part of the supervision and not of the prompt alone: the generator reads and repairs it only because every training example carries one, the same lever that installs new Arabic capabilities in a pretrained speech model (Mattar et al., 2025). Thus, $\text{Makeen}_{\text{Base}}$ and $\text{Makeen}_{\text{+Hint}}$ use the same generator architecture and optimization setup; they differ only in whether the extractor-derived hint is provided as an additional supervised input.

3.3 Confidence-gated Routing

R therefore arbitrates per argument (Algorithm 1). It is gated on *tool agreement*: components disagreeing on the tool have incomparable argument spaces.

A *policy* names the set of routed fields $\mathcal{F}$, and for each $f \in \mathcal{F}$ a confidence threshold θ_f and a mode m_f: REPLACE overwrites a value the generator produced, while PRESENCE lets a confident extraction decide whether the key appears at all. Closed-class spans (country, city, currency, category, …) are first snapped to the value set enumerated from train gold; every field outside $\mathcal{F}$ passes through unchanged. Policies are fitted by greedy forward selection over (field, mode, threshold) on end-to-end ARGEM; details are provided in Appendix B.

4 Experimental Setup

Our experiments proceed in four stages: data curation and tool-declaration refinement, model training, inference-time routing, and generalization to external tools. We first construct quality-controlled training data and refine tool declarations, then train the generator and extractor, apply inference-time routing, while additional translated tools extend the training inventory for evaluating generalization beyond the shared-task setting.

4.1 Data Curation And Tool-Declaration Refinement

We apply LLM-based verification to the 10,550 AISA-ArabicFC training examples, retaining 7,748 high-confidence instances. We then fine-tune an initial Qwen3.5-4B model and use argument-level error analysis to identify ambiguities in the tool declarations. This motivates *Tool-v2*, which refines affected declarations while preserving their original semantics. Detailed filtering criteria, error analysis, and declaration changes are provided in Appendices F and H.

4.2 Generator Training

For all Makeen variants, we fine-tune `Qwen3.5-4B` (Qwen Team, 2026a) with LLaMA-Factory (Zheng et al., 2024), using LoRA and reasoning-trace supervision on the filtered, Tool-v2-enhanced data. Complete optimization and model settings are given in Appendix E.

4.3 Label-Semantic Extractor Setup

The label-semantic extractor E is initialized from `multilingual-e5-base` and fine-tuned on the shared-task training split using the tool and argument descriptions as semantic labels. At inference time, it produces the selected tool, extracted argument values, and per-argument confidence scores c_p. Word-level extraction is decoded at $\theta = 3.0$. The extractor is trained independently of the two generator variants and is shared by $\text{Makeen}_{\text{Base}}$ and $\text{Makeen}_{\text{+Hint}}$.

4.4 Confidence-Gated Routing

Routing is applied at inference time: the output of each Makeen variant is combined with the corresponding extractor output using Algorithm 1. Before routing, closed-class extracted values are snapped at $\sigma = 0.84$ against a value vocabulary derived from training gold. Routing policies are fitted by greedy selection and scored by cross-validation. Additionally, policies are fitted independently for $\text{Makeen}_{\text{Base}}$ and $\text{Makeen}_{\text{+Hint}}$; the resulting policies are listed in Appendix C.

4.5 Generalization To External Tools

An additional experiment was conducted to evaluate generalization beyond the 27 shared-task tools by augmenting training with Arabic translations of Hermes Tool Use Reasoning (interstellarninja, 2025) and Glaive [3]. We translate tool declarations and examples with `Qwen3.6-27B` (Qwen Team, 2026b), retain only single-tool-call examples to match the shared-task setting and generate Arabic reasoning traces for Glaive. To test generalization to unseen tools, we construct a held-out split using lexical grouping, BGE-M3 (Chen et al., 2025) semantic isolation, and an LLM-based cross-pool leakage audit. The resulting split contains 236 examples spanning 126 tools, while the translated training portion contains 5,522 examples across 1,316 tools (Table 6). Detailed construction and leakage-control procedures are provided in Appendix I.

5 Results

Table 1 shows the progression from the zero-shot **Qwen3.5-4B** baseline to the submitted and post-submission **Makeen** variants. Fine-tuning on the filtered and enhanced data, $\text{Makeen}_{\text{Base}}$, increases ARGEM from 51.00% to 81.20% and raises FNACC to 99.63%. Adding extractive hints in $\text{Makeen}_{\text{+Hint}}$ yields a further 1.40-point improvement in ARGEM, accompanied by a 1.28-point decrease in FNACC. Routing offers a further margin in principle for $\text{Makeen}_{\text{Base}}$ and $\text{Makeen}_{\text{+Hint}}$ respectively: under a perfect policy the extractor recovers 7.40 and 5.00 points. Held out by cross-validation, rout-

[3] https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2

#	System	Hint	Routing	ArgEM ↑	FnAcc ↑	ThinkRate ↑	Track B ↑
0	Qwen3.5-4B (baseline)	w/o	w/o	51.00	95.78	22.75	58.78
1	$\text{Makeen}_{\text{Base}}$	w/o	w/o	81.20	**99.63**	100.0	90.49
2*	$\text{Makeen}_{\text{+Hint}}$	w/	w/o	82.60	98.35	100.0	90.80
3*	$\text{Makeen}_{\text{Base}}$	w/o	w/ oracle	*88.60* (+7.40)	99.63	100.0	*94.19*
			w/ cv	81.60 (+0.40)	**99.63**	100.0	90.69
			w/ train	81.40 (+0.20)	**99.63**	100.0	90.59
4*	$\text{Makeen}_{\text{+Hint}}$	w/	w/ oracle	*87.60* (+5.00)	98.35	100.0	*93.31*
			w/ cv	**83.20** (+0.60)	98.35	100.0	**91.11**
			w/ train	82.60 (+0.00)	98.35	100.0	90.80

Table 1: Results on the AISA-ArabicFC development split (545 examples: 500 positive and 45 negative). All values are percentages; ↑ denotes higher is better. Row 0 is the zero-shot Qwen3.5-4B baseline, row 1 is the submitted system, and rows marked * are post-submission experiments. The best result per column is boldfaced.

	Shared-task dev				Unseen-tool dev			
Training data	ArgEM ↑	FnAcc ↑	ThinkRate ↑	Track B ↑	ArgEM ↑	FnAcc ↑	ThinkRate ↑	Track B ↑
None (zero-shot baseline)	51.00	95.78	22.75	58.78	44.49†	98.73	21.19	56.10
Enhanced shared-task data	**81.20**	**99.63**	**100.0**	**90.49**	44.49†	**100.0**	**100.0**	72.24
+ external tools	78.00	96.33	99.63	87.83	**66.53**	**100.0**	**100.0**	**83.26**

Table 2: Effect of translated external-tool augmentation on in-distribution and unseen-tool generalization. $\text{Makeen}_{\text{Base}}$ is fine-tuned on the enhanced shared-task data, while $\text{Makeen}_{\text{+Aug}}$ additionally uses translated external tools; both are evaluated without routing. Results are reported on the AISA-ArabicFC development split and the unseen-tool development split described in Appendix I. The zero-shot Qwen3.5-4B baseline is included for reference and matches Table 1. †Equal as measured: $44.49 = 44.49$.

ing captures only 0.40 and 0.60 of that margin, and choosing the routed arguments from training statistics alone, with nothing tuned on dev, 0.20 and 0.00. **$\text{Makeen}_{\text{Base}}$** configuration was used for the final submission; the hint and routing variants were evaluated post-submission. On the blind test set, the submitted system achieves 82.65% ArgEM, 99.11% FnAcc, and 91.06% Track B, placing our system **9th in Task B**.

Table 2 reveals a clear specialization–generalization trade-off. Training only on the enhanced data, $\text{Makeen}_{\text{Base}}$, gives the strongest performance on the development split, whereas adding translated external tools in $\text{Makeen}_{\text{+Aug}}$ substantially improves performance on unseen tools. In particular, external augmentation increases unseen-tool ArgEM from 44.49% to 66.53% (+22.04 points) and Track B from 72.24% to 83.26% (+11.02 points). The cost on shared-task dev is 2.66 Track B points: ArgEM $81.20 \rightarrow 78.00$ and FnAcc $99.63 \rightarrow 96.33$.

6 Conclusion

We presented **Makeen**, a data-centric approach to Arabic function calling that combines improved supervision with targeted inference-time grounding. Our experiments show that higher-quality training data and tool-declaration refinement substantially strengthen function-calling performance, while semantic extraction and confidence-gated routing improve argument prediction. The oracle results, which provide an upper bound on routing performance, reveal a substantial gap with the observed gains, suggesting room for improvement in routing policy selection and motivating this as a direction for future work. We also investigated generalization to unseen tools and found that translated external tool resources improve transfer, while introducing a trade-off in performance on the shared-task distribution.

These findings highlight the need to balance in-distribution performance with broader tool-level generalization and better understand and optimize this trade-off. The released resources provide a basis for continued research on Arabic function calling, particularly in tool-level generalization and transfer across tool distributions.

Acknowledgments

We gratefully acknowledge Knights Lab at Alexandria University for providing the infrastructure that made the experiments in this work possible.

References

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2025. M3-embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *Preprint*, arXiv:2402.03216.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Kareem Darwish, Nizar Habash, Mourad Abbas, Hend Al-Khalifa, Huseein T. Al-Natsheh, Houda Bouamor, Karim Bouzoubaa, Violetta Cavalli-Sforza, Samhaa R. El-Beltagy, Wassim El-Hajj, Mustafa Jarrar, and Hamdy Mubarak. 2021. A panoramic survey of natural language processing in the arab world. *Communications of the ACM*, 64(4):72–81.

Asım Ersoy, Enes Altinisik, Kareem Mohamed Darwish, and Husrev Taha Sencar. 2025. Tool calling for Arabic LLMs: Data strategies and instruction tuning. In *Proceedings of The Third Arabic Natural Language Processing Conference*, pages 347–358, Suzhou, China. Association for Computational Linguistics.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models.

Mariam Hussein, Sarah Khaled, Marwan Torki, and Nagwa El-Makky. 2023. Alex-U 2023 NLP at WojoodNER shared task: AraBINDER (bi-encoder for Arabic named entity recognition). In *Proceedings of ArabicNLP 2023*, pages 797–802, Singapore (Hybrid). Association for Computational Linguistics.

interstellarninja. 2025. Hermes tool use reasoning. `https://huggingface.co/datasets/interstellarninja/hermes_reasoning_tool_use`. Apache-2.0.

Jie Ma, Miguel Ballesteros, Srikanth Doss, Rishita Anubhai, Sunil Mallya, Yaser Al-Onaizan, and Dan Roth. 2022. Label semantics for few shot named entity recognition. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1840–1851, Dublin, Ireland. Association for Computational Linguistics.

Bassam Mattar, Marwan Torki, and Ayman Khalafallah. 2025. Unlocking additional learning capabilities of Whisper for Arabic language via instruction fine-tuning. In *IEEE/ACS International Conference on Computer Systems and Applications (AICCSA)*. IEEE.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic ai systems.

Qwen Team. 2026a. Qwen3.5: Towards native multimodal agents.

Qwen Team. 2026b. Qwen3.6-27B: Flagship-level coding in a 27B dense model.

Amanda Saunders, Neal Vaidya, Adam Tetelman, and Nik Spirin. 2024. NVIDIA NIM offers optimized inference microservices for deploying ai models at scale. NVIDIA Technical Blog.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.

Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*.

Maolin Wang, Yingyi Zhang, Bowen Yu, Bingguang Hao, Cunyin Peng, Yicheng Chen, Wei Zhou, Jinjie Gu, Chenyi Zhuang, Ruocheng Guo, Wanyu Wang, and Xiangyu Zhao. 2026. Function calling in large language models: Industrial practices, challenges, and future directions. *ACM Computing Surveys*, 58(9):1–37.

An Yang et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyan Luo. 2024. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 400–410, Bangkok, Thailand. Association for Computational Linguistics.

A Example Task Instance

Figure 1 shows an example instance from the development split, as described in §2.1.

TOOLS

- `search_umrah_packages(departure_city, departure_date, duration_days, hotel_rating, num_persons)`
- `search_hotels(check_in, check_out, city, guests, stars)`
- `search_quran(query, search_type, surah_number)`
- `transfer_money(amount, bank_name, currency, recipient_iban, recipient_name)`

USER

أبي أحجز باقة عمرة من الرياض لسبعة أشخاص

GOLD

```
<think>
فهمت أنك ترغب في حجز باقة عمرة من الرياض لسبعة أشخاص. أداة "search_umrah_packages" هي الأنسب لأن
لديها القدرة على البحث عن باقات العمرة المتاحة. المدخلات المتوقعة هي: departure_city=الرياض و
num_persons=7.
</think>
function_call:search_umrah_packages{
  departure_city: الرياض,
  num_persons:7}
```

Figure 1: A Gulf-dialect instance from the development split, showing the four candidate declarations offered to the system, the utterance, and the gold output, all reproduced verbatim. Each declaration is abridged for space to its argument names and the tool's Arabic description, and the string delimiters of the raw serialisation are elided. The value الرياض is copied from the utterance, whereas لسبعة must be normalised to 7; the three remaining tools are distractors, `search_hotels` a close one.

B Confidence-gated Routing Details

Algorithm 1 gives the complete confidence-gated routing procedure. Routing is applied only when the generator and extractor agree on the tool; otherwise, the generator output is returned unchanged. A routed argument is substituted only when its extracted value meets the policy's confidence threshold θ_f. Closed-class spans are snapped using the policy's snapping threshold σ, and any argument not listed in the policy is passed through exactly as the generator produced it. REPLACE overwrites a value the generator already produced; PRESENCE lets a confident extraction decide both whether the key appears and what it holds.

Algorithm 1 Confidence-gated routing

Require: (y_G, a_G), (y_E, a_E, c), policy $(\mathcal{F}, \theta_f, m_f, \sigma)$
if $y_G = \texttt{none}$ **or** $y_G \neq y_E$ **then return** (y_G, a_G)
end if
$a_E \leftarrow \textsc{Snap}(a_E, \sigma)$
for $f \in \mathcal{F}$ **do**
  $ok \leftarrow f \in a_E \wedge c_f \geq \theta_f$
  if $m_f = \textsc{presence}$ **then**
    $a_G[f] \leftarrow a_E[f]$ **if** *ok* **else** drop f
  else if $f \in a_G \wedge ok$ **then**
    $a_G[f] \leftarrow a_E[f]$
  end if
end for
return (y_G, a_G)

Policies are fitted per model; their selected fields, modes, confidence thresholds, and snapping threshold are reported in Appendix C.

A dev example makes the mechanism concrete. For the request below, Makeen$_{\text{+Hint}}$ returns the restaurant name together with the noun مطعم ('restaurant'), whereas E marks the name alone:

	ابي اطلب من مطعم الفخامة، ابغى كبسة ودجاج مشوي
Makeen$_{+\text{Hint}}$	`restaurant` = مطعم الفخامة
E	`restaurant` = الفخامة $(c = 9.4)$
gold	`restaurant` = الفخامة

Both components agree on `order_food`, and `restaurant` is a routed field, so the extraction replaces the generated value and the row becomes an exact match. The same row's `items` argument is left untouched, since it is not a routed field.

C Routing Policies

Table 1 reports three ways of choosing a routing policy. This section gives each of them, using the confidence threshold θ_f and the REPLACE/PRESENCE modes defined in §B. In every case the snapping threshold is $\sigma = 0.84$.

C.1 Train-Derived Selection

No search on the development split. An argument is routed when its gold value is a verbatim span of the query in at least 95% of its *training* occurrences; all routed arguments use REPLACE with $\theta_f = 0$. The rule selects the same 13 arguments for both systems and gives the 0.20 and 0.00 margins.

argument	% direct	n	argument	% direct	n
`visa_number`	100.0	494	`text`	98.5	547
`insurance_number`	100.0	397	`city`	98.4	2802
`recipient_iban`	100.0	278	`years_of_service`	97.4	491
`id_number`	99.8	507	`product_value`	97.0	508
`query`	99.4	506	`destination_country`	95.8	481
`salary`	99.4	491	`departure_city`	95.1	473
`iqama_number`	99.1	529			

Table 3: Arguments selected from training statistics alone. *% direct* is the share of training occurrences whose gold value appears verbatim in the query; n is the number of occurrences.

C.2 Cross-Validated Fold Policies

Cross-validation fits one policy per fold, so the reported figure is not produced by a single policy: the five held-out fold predictions are stitched together and scored once.

Makeen$_{\text{Base}}$ out-of-fold 81.60 (+0.40)				
fold	fit/test	argument	mode	θ_f
0	433/112	category	REPLACE	0.0
		city	REPLACE	0.0
		country	REPLACE	5.0
		currency	REPLACE	7.0
		medication_name	REPLACE	0.0
		query	REPLACE	0.0
		restaurant	REPLACE	0.0
		type	PRESENCE	0.0
1	435/110	city	REPLACE	0.0
		country	REPLACE	5.0
		currency	REPLACE	7.0
		medication_name	REPLACE	0.0
		query	REPLACE	0.0
		type	PRESENCE	0.0
2	435/110	bank_name	PRESENCE	7.0
		city	REPLACE	0.0
		country	REPLACE	5.0
		destination_country	PRESENCE	7.0
		medication_name	REPLACE	7.0
		query	REPLACE	0.0
		recipient_iban	PRESENCE	5.0
		restaurant	REPLACE	0.0
		text	REPLACE	0.0
		type	PRESENCE	0.0
3	437/108	country	REPLACE	5.0
		currency	REPLACE	7.0
		date	REPLACE	0.0
		medication_name	REPLACE	0.0
		restaurant	REPLACE	0.0
		text	REPLACE	0.0
		type	PRESENCE	0.0
4	440/105	bank_name	PRESENCE	7.0
		city	REPLACE	0.0
		country	REPLACE	5.0
		medication_name	REPLACE	0.0
		query	REPLACE	0.0
		recipient_iban	PRESENCE	5.0
		restaurant	REPLACE	0.0
		text	REPLACE	0.0
		type	PRESENCE	0.0

Makeen$_{+\text{Hint}}$ out-of-fold 83.20 (+0.60)				
fold	fit/test	argument	mode	θ_f
0	436/109	`category`	REPLACE	7.0
		`city`	REPLACE	0.0
		`destination_country`	PRESENCE	7.0
		`medication_name`	REPLACE	0.0
		`query`	REPLACE	0.0
		`recipient_iban`	PRESENCE	5.0
		`restaurant`	REPLACE	0.0
		`type`	PRESENCE	0.0
1	437/108	`city`	REPLACE	0.0
		`country`	REPLACE	5.0
		`destination_country`	PRESENCE	0.0
		`medication_name`	REPLACE	0.0
		`query`	REPLACE	0.0
		`recipient_iban`	PRESENCE	5.0
2	436/109	`category`	REPLACE	7.0
		`city`	REPLACE	0.0
		`country`	REPLACE	5.0
		`destination_country`	PRESENCE	7.0
		`medication_name`	REPLACE	0.0
		`restaurant`	REPLACE	0.0
		`type`	PRESENCE	0.0
3	435/110	`category`	REPLACE	0.0
		`city`	REPLACE	0.0
		`destination_country`	PRESENCE	7.0
		`medication_name`	REPLACE	0.0
		`query`	REPLACE	0.0
		`recipient_iban`	PRESENCE	5.0
		`restaurant`	REPLACE	0.0
4	436/109	`category`	REPLACE	7.0
		`city`	REPLACE	0.0
		`country`	REPLACE	5.0
		`destination_country`	PRESENCE	7.0
		`medication_name`	REPLACE	0.0
		`query`	REPLACE	0.0
		`recipient_iban`	PRESENCE	5.0
		`restaurant`	REPLACE	0.0

Table 4: Per-fold routing policies. Folds are stratified by gold tool; *fit/test* gives the number of rows the policy was fitted on and scored on. The selected set varies from fold to fold – `city` and `medication_name` appear in every fold, while `country`, `type` and `bank_name` appear in some only – and thresholds move with it, which is why no single policy stands behind the cross-validated figure.

D Translation Prompts

Prompts used to translate the data of §4.5: Figure 2 for a tool declaration and Figure 3 for a sample, given its already-translated declaration as context; Figure 4 replaces rule 2 of the sample prompt for Glaive.

```
You are a precise dataset translator for a tool-calling (function-calling) dataset.
Translate ONLY the human-readable text of the given tool DECLARATION from English
into Arabic.

Rules:
1. Keep "name" UNCHANGED (it is an API identifier).
2. Keep every parameter KEY UNCHANGED (they are API identifiers).
3. Keep all "type" values UNCHANGED.
4. Translate the "description" fields (the tool description and each parameter
   description) into natural Arabic.
   - If a description embeds enum/identifier tokens (e.g. "Supported values:
     driving, walking, transit"), keep those tokens UNCHANGED and translate only
     the surrounding words.
5. Preserve the JSON structure exactly (same keys, same nesting).
6. Output ONLY the translated declaration as a single valid JSON object, no
   markdown, no commentary.

Do not justify or explain your response. Only deliver your response in the JSON.

INPUT:
{<tool declaration>}

OUTPUT JSON:
```

Figure 2: Declaration prompt.

```
You are a precise dataset translator for a tool-calling (function-calling) dataset.
Translate the given fields from English into Arabic.
The tool declaration is ALREADY translated and provided under
"translated_declaration_context" only as context. Do NOT translate it and do NOT
output it.

Follow these rules EXACTLY:
1. Translate "human" (the user query) into natural Arabic. You have to
   translate everything, including names.
2. Translate "think" (the reasoning) into natural Arabic.
   - Make the translated version shorter.
3. For "tool_call":
   - Keep "name" UNCHANGED (it is an API identifier).
   - Keep every argument KEY UNCHANGED (they are API identifiers).
   - For each argument VALUE:
     * If the value is natural-language text taken from the user query, translate
       it so that the translated value is an EXACT substring of your translated
       "human" field, copy the same wording verbatim, do NOT rephrase, do NOT add
       or drop words. Alignment must be exact.
4. Output ONLY a single valid JSON object, no markdown, no commentary, with keys:
   "human", "think", "tool_call".
   "tool_call" must be a nested object with "name" and "arguments" (arguments as
   a nested dict).

Do not justify or explain your response. Only deliver your response in the JSON.

INPUT:
{
  "human": "<user query>",
  "think": "<reasoning>",
  "tool_call": {"name": "<tool>", "arguments": {...}},
  "translated_declaration_context": {<declaration already translated>}
}

OUTPUT JSON:
```

Figure 3: Sample prompt (Hermes).

Training parameter	Value
Base model	`Qwen/Qwen3.5-4B`
Framework	LLaMA-Factory
Fine-tuning	LoRA
Template	`qwen3_5`
Attention	FlashAttention-2
Learning rate	1×10^{-4}
Epochs	3
Per-device batch size	16
Gradient accumulation	2
Warmup ratio	0.03
Quantization	4-bit
LoRA rank	16
LoRA α	32
LoRA dropout	0.05
enable_reasoning	True

Table 5: Fine-tuning configuration.

Split	Samples	Tools
Training		
AISA-ArabicFC, Tool-v2-enhanced	7,748	27
Hermes + Glaive, translated	5,522	1,316
all	13,270	1,337
Development, unseen tools		
Hermes + Glaive, translated	236	126

Table 6: MAKEEN$_{+\text{AUG}}$ data. *Samples* counts examples after filtering, *Tools* distinct declared tools; the total counts shared function names once. train $\cap$ dev $= \emptyset$.

```
2. For "think" (the reasoning):
   - GENERATE long, natural Arabic "think" that reasons about which tool to
     call and why, based on the "human" query, the "tool_call", and the provided
     translated declaration context. Keep it concise (2-3 sentences). Do NOT leave
     it empty.
```

Figure 4: Rule replacing rule 2 of Figure 3 for Glaive, whose examples carry no reasoning trace.

E Training Configuration

We fine-tune `Qwen/Qwen3.5-4B` (Qwen Team, 2026a) using LLaMA-Factory (Zheng et al., 2024). Preliminary evaluation on the AISA-ArabicFC data indicated that the base model already exhibited basic function-calling ability, motivating task-specific adaptation to the dataset's tool declarations and annotation conventions. The complete training configuration is summarized in Table 5.

For the additional generalization experiment of Section 4.5, the shared-task training pool is augmented with translated external tools to train MAKEEN$_{+\text{AUG}}$, and a held-out unseen-tool development split is used for evaluation. Table 6 reports the size of each.

Training used NVIDIA RTX PRO 6000 and RTX 4090 GPUs with 97GB and 24GB memory, respectively.

F Data Quality Control

F.1 Overview

AISA-ArabicFC data was subjected to strict LLM-based quality control. The evaluator assessed each annotation against the available system prompt, user request, and tool declaration. Rejected cases were subsequently inspected to recover valid examples that had been conservatively rejected.

Of the 10,550 initial examples, 6,391 were accepted by the LLM-based evaluator. Manual inspection of rejected cases recovered valid examples, yielding a final pool of 7,748 examples. The resulting dataset is therefore treated as high-confidence rather than error-free, with filtering prioritizing supervision quality and consistency over dataset size.

Group	Error type	Description
Function selection	`wrong_function`	A different tool is selected from the one required by the request.
	`no_function_needed`	A tool is called although the request does not require one.
Argument extraction	`missing_parameter`	A required argument is omitted.
	`extra_parameter`	An unnecessary argument is added.
	`hallucinated_value`	A value not supported by the system prompt or user request is introduced.
	`wrong_parameter_name`	The argument name does not match the tool declaration.
	`wrong_parameter_type`	The argument type violates the tool declaration.
	`wrong_argument_value`	The extracted value does not match the intended annotation.
Representation	`translated_parameter`	A user-provided value is unnecessarily translated.
	`rephrased_parameter`	The user's wording or representation is unnecessarily changed.
	`unsupported_assumption`	Information not established by the input is inferred or assumed.
Schema/supervision	`invalid_format`	The function call does not follow the required call format.
	`schema_violation`	The annotation violates a schema constraint not captured by a more specific error category.
	`ambiguous_request`	The input does not provide sufficient information for a unique decision.
	`multiple_errors`	Multiple substantive errors are present and no single primary error adequately characterizes the annotation.

Table 7: Error taxonomy used by the LLM-based evaluator for training-data quality control and function-calling error analysis.

F.2 Evaluation Criteria And Error Taxonomy

The LLM-based evaluator checks five complementary dimensions: function selection, argument completeness, value fidelity, schema compliance, and function-call format. It returns a binary decision and, for rejected annotations, assigns one primary error reason. These reasons are grouped into function selection, argument extraction, representation, and schema/supervision categories, as summarized in Table 7. The complete evaluation prompt is provided in Appendix G.

F.3 Evaluator Models

The LLM-based evaluator was implemented using five LLMs accessed through NVIDIA NIM APIs (Saunders et al., 2024). Multiple evaluator models were used during large-scale filtering to reduce operational bottlenecks and API rate-limit constraints. The models were

- `nvidia/nemotron-3-ultra-550b-a55b`
- `z-ai/glm-5.1`
- `deepseek-ai/deepseek-v4-pro`
- `moonshotai/kimi-k2.6`
- `stepfun-ai/step-3.7-flash`

F.4 Representative Data-Annotation Errors

Table 8 presents representative errors identified in the training data annotations by the LLM-based evaluator. Each example includes only the context necessary to interpret the error, with the erroneous function call or argument highlighted alongside the assigned error category and diagnosis.

Error	Example
`hallucinated_value`	**System prompt:** YYYY-MM-DDTHH:MM:SS: 2024-07-23T09:23:15 التاريخ والوقت الحالي بصيغة اليوم هو الثلاثاء أنت نموذج يمكنه استدعاء الوظائف التالية **User:** عندي رحلة على بيروت، في فنادق مقترحة؟ `check_in = 2023-11-01` **Issue:** The date was neither provided by the user nor supported by the temporal context present in the system prompt.
`missing_parameter`	**User:** ازاي أحسب زكاة الذهب لو عندي 50 جرام؟ **Issue:** The reference annotation omitted the user-specified weight unit. `weight_unit = gram`
`no_function_needed`	**User:** متى يمكنني حجز موعد مع طبيب قلب في الرياض؟ The reference annotation calls: `book_doctor_appointment` **Issue:** The request asks when an appointment can be booked rather than asking to perform a booking.
`wrong_function`	**User:** ممكن أسأل عن دكاترة عيون في القاهرة؟ The reference annotation calls: `book_doctor_appointment` **Issue:** The request is a doctor-search or information request rather than an appointment-booking request.
`translated_parameter`	**User:** ممكن حجز مع دكتور نساء وتوليد في الزقازيق يوم الجمعة؟ **Issue:** The reference annotation translated the user-provided temporal value instead of preserving its original representation. `date = Friday`
`wrong_parameter_name`	**User:** كم نسبة الزكاة على الذهب عيار ٢٤ اذا عندي ٥٠٠ جرام؟ The reference annotation contains: `currency = gram` **Issue:** The annotation uses `currency`, whereas the tool declaration defines the argument as `weight_unit`. Argument names must exactly match the tool declaration; therefore, this is a `wrong_parameter_name` error.

Table 8: Representative training-annotation errors identified by the LLM-based evaluator. Each example shows the relevant user request and erroneous annotation, together with the assigned error category and diagnosis.

G Complete Filtering Prompt

For reproducibility, the evaluator prompt is provided below. The placeholders `{{tools}}`, `{{system_prompt}}`, `{{user_query}}`, and `{{function_call}}` are replaced with the corresponding example fields.

```
You are an expert evaluator of function-calling annotations.

Your task is to determine whether the provided function call is a perfect
annotation of the user's request.

Available tools:

{{ tools }}

Evaluation criteria:

1. The selected function is the single best choice among the available tools.
2. A function should only be called if the user's request actually requires one.
3. If no available tool should be called, any function call is incorrect.
4. Every required parameter is present.
5. No unnecessary or hallucinated parameters are included.
6. Every parameter value is exactly supported by the original system prompt or the
   user's request.
7. Missing information must not be invented or assumed.
8. Parameter names exactly match the selected tool's schema.
9. Parameter values have the correct types according to the tool schema.
10. The function call follows the required function-call format.
11. The function call fully conforms to the selected tool's schema.
12. Ignore insignificant differences in whitespace or formatting that do not
    affect the meaning or structure of the function call.
13. Parameter values must preserve the user's original wording unless the tool
    schema explicitly requires normalization (e.g., enum values, ISO dates, standardized
    formats).
14. Translating, paraphrasing, summarizing, correcting spelling, expanding
    abbreviations, or otherwise modifying a parameter value is considered an
    error unless explicitly required by the tool schema.

Decision rule:
- Return `label = 1` ONLY if every criterion is unquestionably satisfied.
- If there is any ambiguity, unsupported assumption, missing information,
  incorrect function, incorrect parameter, invalid extraction, schema violation,
  formatting violation, translation, paraphrasing, or uncertainty, return `label = 0`.

Reason codes:
If `label` is `0`, choose ONLY ONE primary reason from the following list:

[wrong_function, no_function_needed, missing_parameter, extra_parameter, translated_parameter,
rephrased_parameter, wrong_parameter_name, wrong_parameter_type, wrong_argument_value,
hallucinated_value, invalid_format, schema_violation, unsupported_assumption, ambiguous_request,
multiple_errors]

If `label` is `1`, use: correct

Output format:
Return ONLY valid JSON.

Correct annotation:
{ "label": 1, "reason": "correct"}

Incorrect annotation:
{ "label": 0, "reason": "<reason_code>"}

Do not output markdown.
Do not output explanations.
Do not output any additional text.

Original system prompt:

{{ system_prompt }}

Original user request:

{{ user_query }}

Function call format:

The function call is NOT JSON. It always follows this XML-like structure:
<tool_call><function=function_name><parameter=parameter_name>parameter_value</parameter>...</function></tool_call>

Evaluation notes:

- The function name is specified in the opening `<function=...>` tag.
- Each parameter is represented by a `<parameter=...>` tag.
- Parameter values are plain text between the opening and closing parameter tags.
- The order of parameters is not important unless explicitly required by the tool schema.
- Ignore insignificant whitespace and line breaks.
- Evaluate semantic correctness rather than exact formatting, provided the
  structure remains valid.
- Do not infer or guess missing information.
- Reject any function call containing unsupported, hallucinated, or incorrectly extracted values.
- Validate the function call against the provided tool schema, not against the example above.
```

```
Function call to evaluate:

{{ function_call }}

Result:
```

Figure 5: Full evaluation prompt used by the LLM-based evaluator to assess training annotations against the system prompt, user request, and tool declaration.

H Tool-Declaration Refinement And Data Consistency Analysis

We conducted an error analysis of function-calling predictions from an initial Qwen3.5-4B model fine-tuned on the pre-processed training data. The analysis focused on argument-level errors and identified recurring cases in which failures were associated with ambiguity, incompleteness, or insufficient specificity in the corresponding tool declarations.

Based on these observations, we introduced **Tool-v2** by refining affected tool declarations while preserving their original functionality and task semantics. The refinements target four recurring sources of ambiguity: (i) preservation of user-provided values, (ii) incomplete categorical value enumerations, (iii) ambiguity between semantically related arguments, and (iv) unclear extraction boundaries. The corresponding changes are illustrated in Sections H.1–H.4.

The analysis also revealed substantial variation in the surface-form representation of date arguments across the reference annotations. Because this variation is present in the annotations themselves, it cannot be resolved through tool-declaration refinement alone. We therefore retain date representation as a data-level ambiguity rather than imposing an arbitrary canonical format (Section H.5).

This data-level ambiguity is distinct from the Tool-v2 refinements, which modify only the tool descriptions provided to the model while preserving the underlying tool functionality and task semantics.

H.1 Preserving User-Provided Representations

A recurring error involved unnecessary transformation of values explicitly provided by the user, particularly names of cities and countries. To make the expected behavior unambiguous, the corresponding argument descriptions were revised to require preservation of the user's original representation.

Language preservation

Tool-v1		Tool-v2
اسم المدينة بالعربية	⟶	اسم المدينة بالعربية، وإذا أدخله المستخدم بلغة أخرى فاستخدمه كما هو
اسم الدولة بالعربية	⟶	اسم الدولة بالعربية، وإن ذكرها المستخدم بلغة أخرى فاستخدمها كما هي

Refinement: Explicitly preserve user-provided values rather than implicitly converting them to Arabic or English.

H.2 Completing Categorical Value Enumerations

Several tool declarations listed only a subset of the categorical values observed in the reference annotations. Such incomplete enumerations can make the declaration inconsistent with valid reference values and may reduce the model's ability to associate an argument with its supported value space. We therefore expanded the affected enumerations to include additional values supported by the data.

Categorical value coverage

Tool-v1	Tool-v2
`money, gold, silver, trade_goods, stocks`	`cash, gold, fitr, trade, crops, silver, general, livestock, realestate, shares, salary`
`resignation, termination, retirement`	`dismissal, resignation, end_of_contract, retirement, unfair_dismissal, economic, mutual_consent, disciplinary`
`text, topic, surah`	`text, topic, surah, tafseer, verse`

Refinement: Complete the enumeration using valid values supported by the training data, rather than restricting the model to an incomplete subset.

H.3 Clarifying Argument Semantics

Some errors involved confusion between arguments with closely related semantics. We therefore refined the affected descriptions to make the intended meaning and admissible value types explicit.

Argument semantics

Tool-v1	Tool-v2
`recipient_name`	
اسم المستفيد	اسم المستفيد. يجب أن يكون اسم شخص حقيقي وليس وصفا مثل "صديقي" أو "أخي"
`doctor_name`	
اسم الطبيب (اختياري)	اسم الطبيب ويجب أن يكون اسم شخص حقيقي (اختياري)
`product_value`	
قيمة المنتج	قيمة المنتج (سعره الإجمالي أو المبلغ المدفوع لشرائه)

Refinement: Disambiguate closely related concepts by explicitly stating what qualifies as a valid argument value.

H.4 Clarifying Extraction Boundaries

We also observed cases in which the predicted argument span contained information beyond the intended value. Where the reference annotations supported a consistent extraction rule, the corresponding descriptions were refined to state the required boundary explicitly.

	ArgEM↑	FnAcc↑	ThinkRate↑	Track B↑
$\text{Makeen}_{\text{Base}}$	76.60 → **81.20** (+4.60)	94.68 → **99.63** (+4.95)	100.0 → 100.0 (+0.00)	86.70 → **90.49** (+3.79)

Table 9: Effect of tool-description refinement on $\text{Makeen}_{\text{Base}}$. Each cell reports *Tool-v1* → *Tool-v2*, with the absolute change shown in parentheses. Tool-v1 uses the original shared-task tool descriptions, whereas Tool-v2 uses the refined descriptions.

Extraction boundaries

Tool-v1		Tool-v2
`items`		
قائمة الطعام	→	قائمة الطلب كاملة مع كل التفاصيل كما هي بدون اختصار أو حذف
`text`		
النص المراد ترجمته	→	النص المراد ترجمته فقط، يجب استخراج النص المطلوب للترجمة دون تضمين كامل الاستعلام

Refinement: Specify the intended argument span instead of leaving the extraction boundary implicit.

H.5 Date Representation Ambiguity

The same analysis revealed substantial variation in the surface form of date arguments such as `check_in` and `check_out`. Observed representations include ISO-formatted dates (`2024-11-10`), day–month expressions (١ مارس), Arabic-Indic numerals (٢٠ أكتوبر), and relative temporal expressions (بداية الشهر القادم).

Because these variations occur in the reference annotations themselves, they cannot be resolved through tool-declaration refinement alone. Imposing a canonical representation at the tool-declaration level without first resolving the underlying annotation inconsistency would introduce an additional assumption not supported by the data.

We therefore did not impose a canonical date format in Tool-v2. Instead, we treat date representation as a data-level ambiguity that should be resolved at the annotation level before a canonical representation is encoded in future tool specifications.

H.6 Empirical Impact Of Tool-Declaration Refinement

To quantify the effect of the tool-declaration refinements, we evaluated $\text{Makeen}_{\text{Base}}$ using two versions of the tool descriptions: (i) the original descriptions provided with the shared-task data (Tool-v1), and (ii) the refined descriptions introduced in Tool-v2. All other aspects of the system, including the model, training procedure, and evaluation setting, were kept unchanged.

As shown in Table 9, Tool-v2 yields consistent improvements in both function selection and argument extraction. In particular, ArgEM increases by 4.60 percentage points, while FnAcc increases by 4.95 percentage points. Because the comparison changes only the tool descriptions while keeping the model and evaluation setting fixed, these results provide evidence that improving the specificity and consistency of tool declarations can materially affect function-calling performance.

I Construction Of The Unseen-Tool Development Split

We construct the external-data train/dev split to evaluate generalization to functions that are not observed during training. The procedure operates over the combined inventory of shared-task and translated tools. First, we group functions using shared name tokens and exclude groups containing shared-task functions.

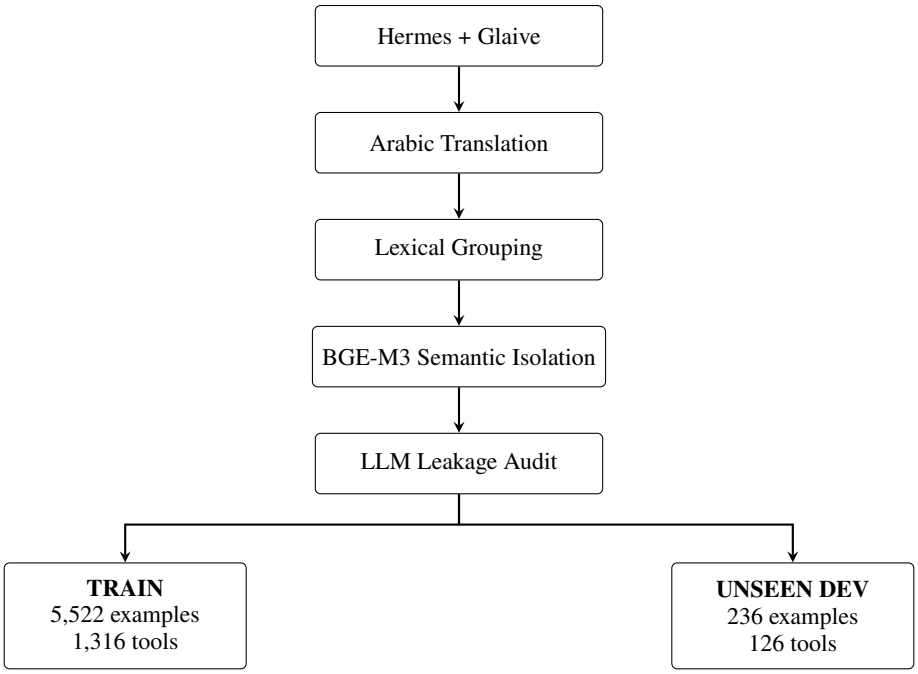


Figure 6: Construction of the external-tool training and unseen-tool development sets. Hermes and Glaive examples are translated into Arabic, grouped lexically, filtered using BGE-M3 semantic similarity to isolate tool groups, and subsequently audited with an LLM to reduce potential cross-split leakage. The final split is performed at the tool level, with 1,316 training tools and 126 unseen development tools.

These lexical groups serve only as candidate families for the subsequent semantic filtering. We then encode each tool using `BAAI/bge-m3` (Chen et al., 2025), representing the function name, description, and argument descriptions. A candidate held-out group is retained only when every function in the group has cosine similarity below 0.75 to every function outside the group in the combined tool inventory.

The resulting candidates are subjected to an exhaustive cross-pool semantic audit using GPT-5.6 Luna as an LLM judge. The judge receives the final TRAIN and DEV tool declarations as separate pools and identifies residual semantic overlap between the two inventories. We additionally verify function-level disjointness and candidate-pool separation before constructing the final examples. Each example is assigned according to its gold function, and candidate tools are drawn exclusively from the same split.

The final external-data training split contains $5{,}522$ examples spanning $1{,}316$ tools, while the held-out development split contains 236 examples spanning 126 tools. The shared-task development split is retained separately for evaluation on the original 27-tool inventory.

J Makeen Pipeline

Figure 7 illustrates the complete Makeen pipeline, from the generator and label-semantic extractor to hint construction and confidence-gated routing.

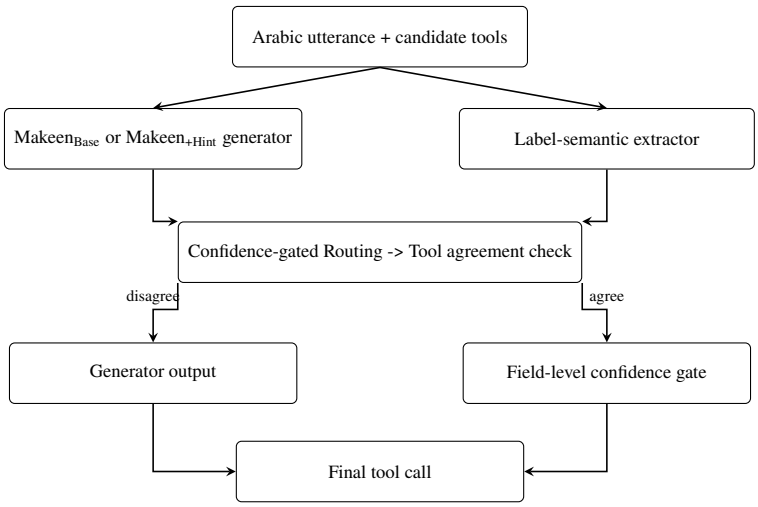


Figure 7: Overview of the Makeen pipeline. The submitted Makeen$_{\text{Base}}$ generator is extended with label-semantic extraction for Makeen$_{\text{+Hint}}$ and with confidence-gated routing at inference time.